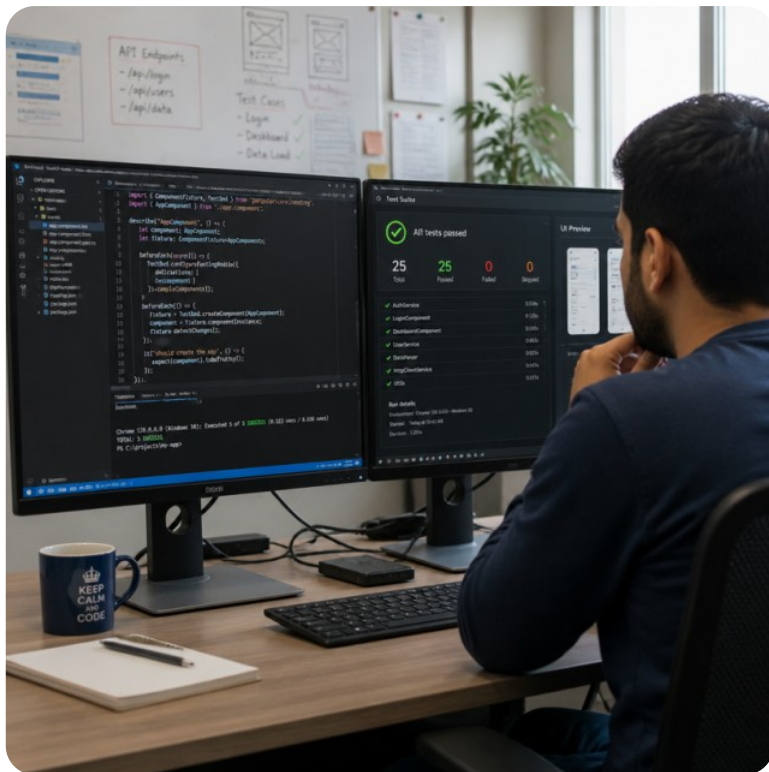




Fundamentos de Testes de Software

Domine qualidade, STLC e a Pirâmide de Testes no desenvolvimento frontend moderno.

O Custo de um Clique Quebrado

Em 1996, o foguete Ariane 5 explodiu 40 segundos após o lançamento por causa de uma conversão aritmética não testada. No frontend, um simples erro em um botão pode impedir milhares de compras em segundos.



Objetivos da Aula

O que você vai dominar hoje

Compreender a relevância do QA e as 5 fases do STLC no ciclo de software.

Diferenciar testes unitários, de integração e E2E aplicando a Pirâmide de Testes.

Analisar falhas reais e propor estratégias eficientes de automação.



Ponto-chave

Testar não é encontrar erros depois de pronto: é construir confiabilidade contínua desde o primeiro commit.

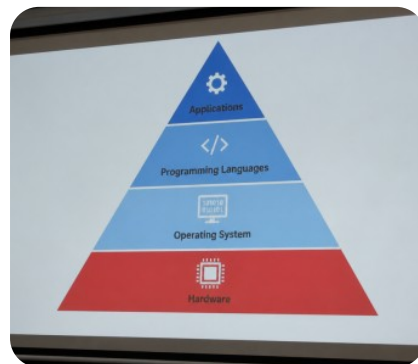
Vocabulário Essencial de QA



Qualidade - Conformidade do sistema com requisitos e expectativas dos usuários.

```
38 | TEST_CASE("Test factorial values", "[factorial]") {
39 |     REQUIRE(factorial(0) == 1);
40 |     REQUIRE(factorial(1) == 1);
41 |
42 |     SECTION("Small numbers") {
43 |         REQUIRE(factorial(2) == 2);
44 |         REQUIRE(factorial(3) == 6);
45 |         REQUIRE(factorial(4) == 24);
46 |         REQUIRE(factorial(5) == 120);
47 |     }
48 |     SECTION("Compare expected and actual") {
49 |         int expected = 720;
50 |         int received = factorial(6);
51 |         // Check that the received value matches expected
52 |         REQUIRE(expected == received);
53 |         // Additional check with different values
54 |         expected = 5040;
55 |         received = factorial(7);
56 |         REQUIRE(expected == received);
57 |     }
58 | }
```

Assertividade - Capacidade de validar com precisão se o resultado obtido é igual ao esperado.



Pirâmide - Modelo que balanceia volume, velocidade e custos de diferentes tipos de testes.



Automação - Uso de scripts para executar rotinas de testes repetíveis sem intervenção humana.

O Papel Estratégico do QA

MERCADO

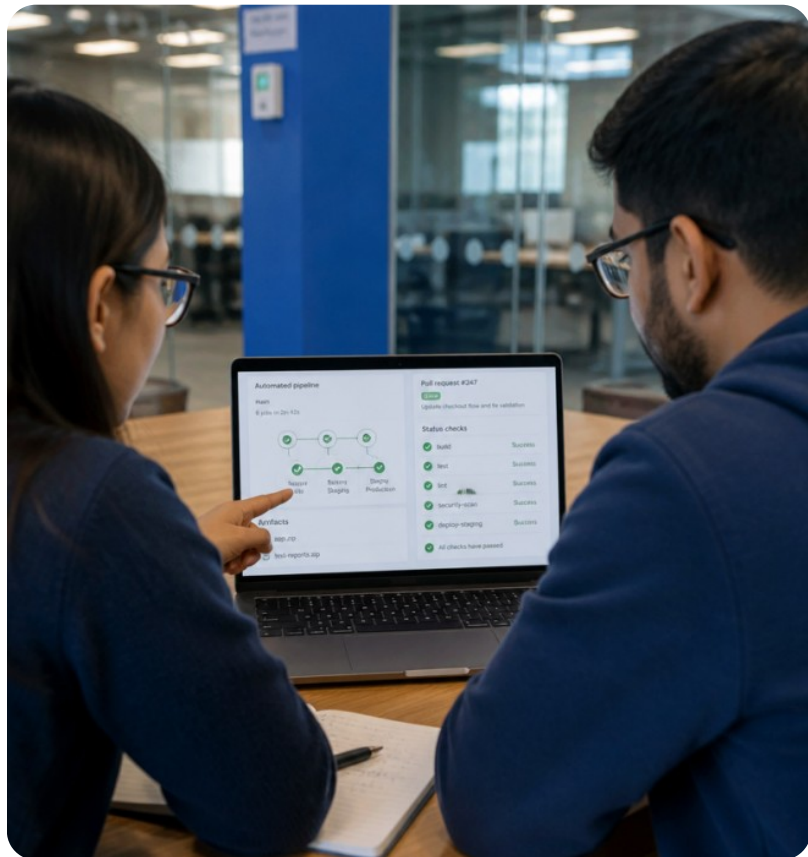
Evolução: Da revisão manual à engenharia

QA deixou de ser apenas cliques manuais pré-lançamento. Hoje, integra código, arquitetura ágil e pipelines CI/CD. Times modernos realizam deploys diários com suites de testes automatizados.



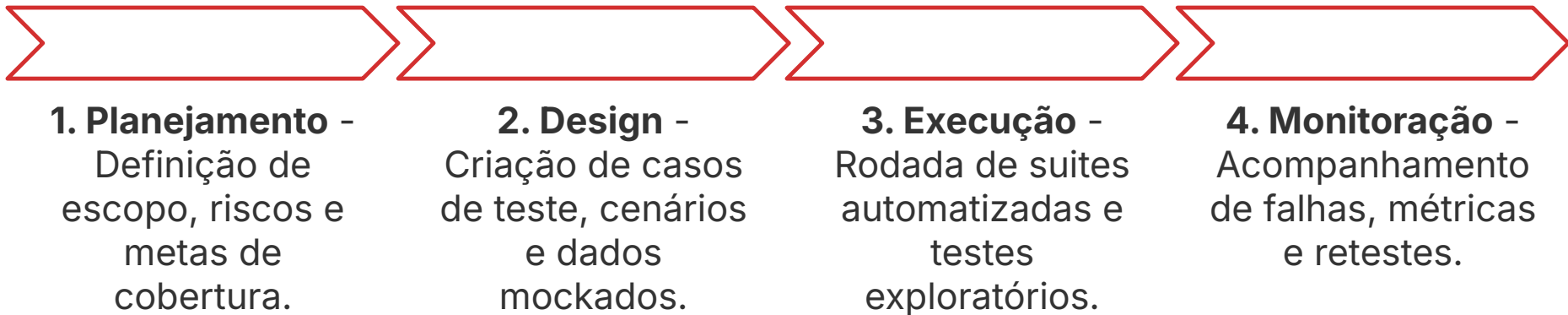
Lembre-se

QA não é isolado no fim da esteira: é responsabilidade compartilhada por toda a equipe.



As 5 Fases Fundamentais do STLC

O STLC define uma jornada sistemática para garantir conformidade técnica.



STLC na Prática de Frontend



METODOLOGIA

Estruturação antes de codificar

No frontend, o **Design** planeja interações, estados assíncronos e falhas de rede. A **Monitoração** avalia intermitência (flakiness) e tempo de suite, mantendo feedback abaixo de 10 min.

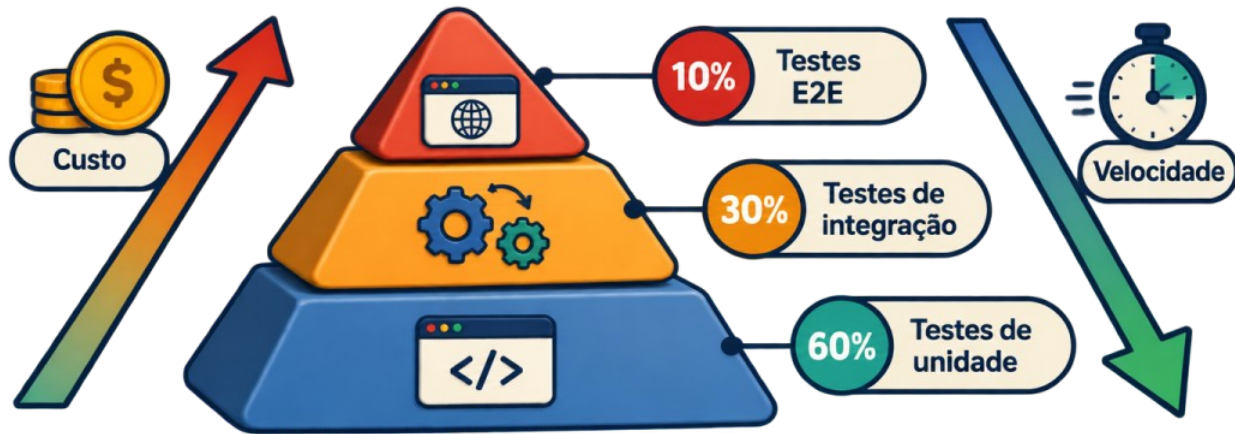


Exemplo

No checkout, mapeie timeouts de gateway e erros de cartão.

A Anatomia da Pirâmide de Testes

Criada por Mike Cohn, a pirâmide equilibra velocidade, isolamento e custo.



Lembre-se

A proporção 60-30-10 assegura validações rápidas na base e poucos E2E no topo.

Testes Unitários: A Base Firme

NÍVEIS DE TESTE

Isolamento e rapidez

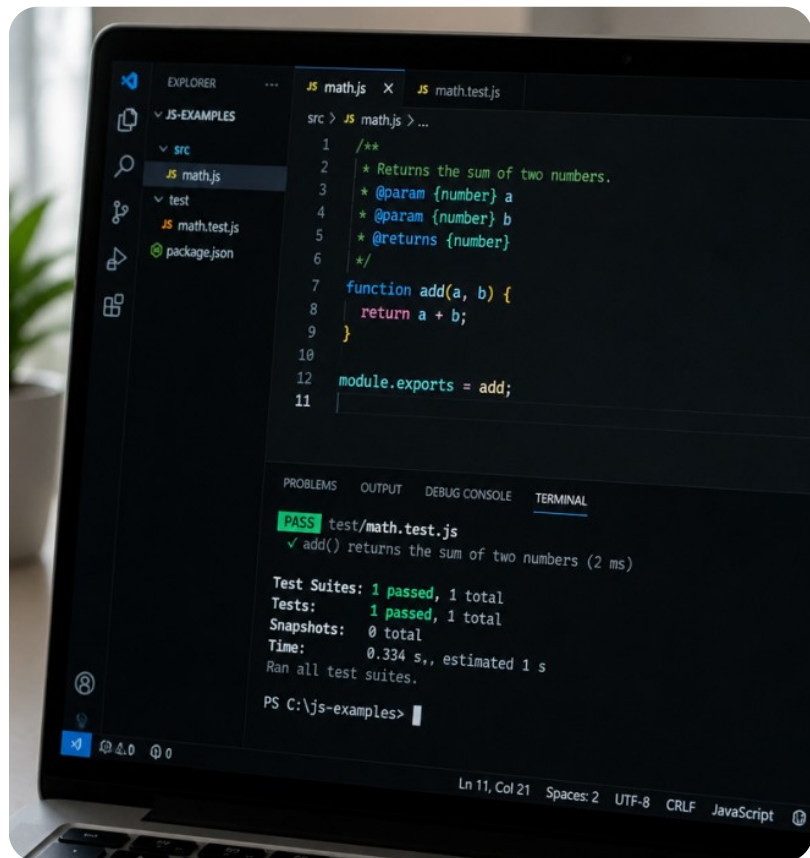
Validam a menor fração testável: funções puras, validadores ou utilitários.

- Execução < 100ms.
- Sem navegadores ou bancos.
- Precisão na falha lógica.

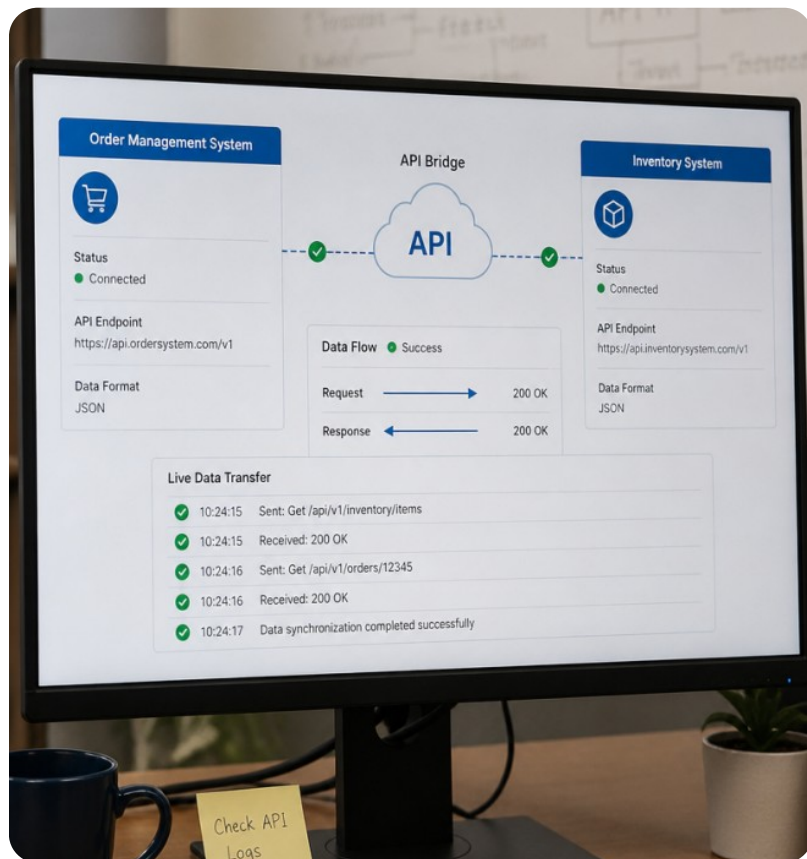


Exemplo

Validar `formatarMoeda(1500) == 'R$ 1.500,00'` sem carregar DOM.



Testes de Integração: Conexões



NÍVEIS DE TESTE

Integração de Componentes

Validam interações entre módulos e contratos de API. Peças unitárias podem falhar ao se conectar.

- Duração: 100ms a 1s.
- Usam mocks/stubs para simular APIs.
- Garantem envio correto de payloads.



Atenção

Peças isoladas perfeitas podem falhar ao serem integradas.

Testes End-to-End (E2E): O Topo

NÍVEIS DE TESTE

Visão do usuário final

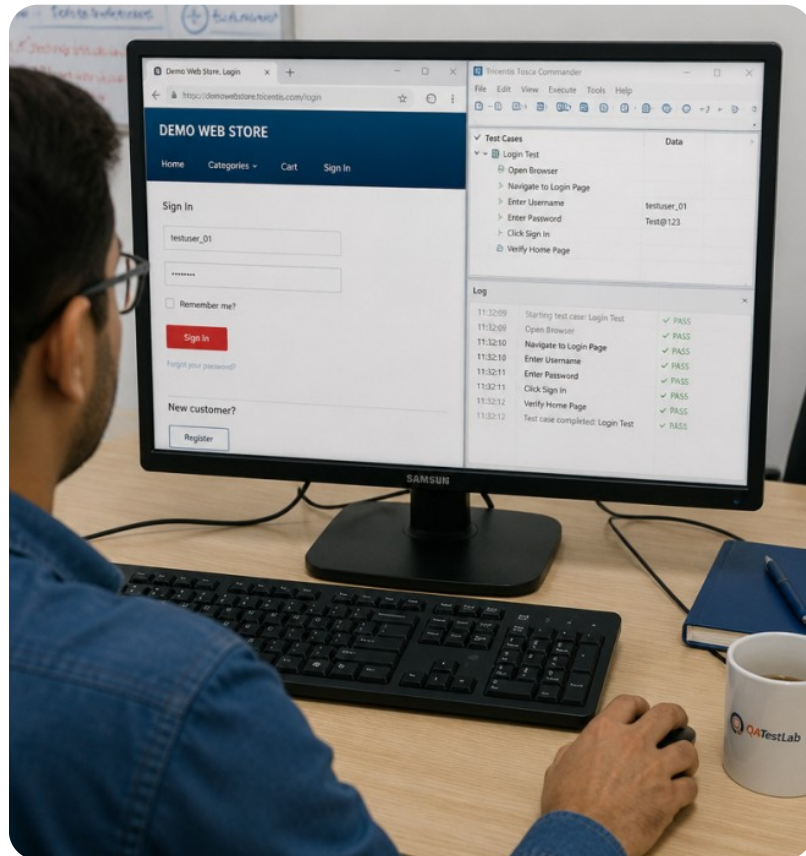
Simulam navegadores reais em fluxos completos, do login ao pagamento.

- Duração: 5 a 30 segundos.
- Testam frontend, rede e backend.
- Sensíveis a latência e CSS.



Ponto-chave

Cubra apenas fluxos críticos (happy paths).



Comparativo dos Três Níveis

Unitários (60%)

Escopo: Funções isoladas

Velocidade: <100ms

Custo: Mínimo

Isolamento: Total

Exemplo: Cálculo de desconto



Exemplo

Milhares rodam em segundos.

Integração (30%)

Escopo: Múltiplos módulos

Velocidade: 100ms-1s

Custo: Médio

Isolamento: Parcial (Mocks)

Exemplo: Form + Validação



Ponto-chave

Validam DOM e estado.

End-to-End (10%)

Escopo: Sistema total

Velocidade: 5s-60s

Custo: Alto

Isolamento: Nenhum

Exemplo: Jornada compra



Atenção

Sujeitos a lentidão e falhas.

O Perigo da Pirâmide Invertida

O Sorvete de Testes

Excesso de testes E2E e falta de unitários cria um cone de sorvete frágil:

- Pipelines levam horas.
- Testes falham aleatoriamente (flakiness).
- Alertas de erro são ignorados.



Atenção

Manutenção supera o tempo de novas features.

O Caso Airbnb

O Airbnb tinha 80% de testes E2E: 6h de execução e 35% de falhas intermitentes.

Ao adotar o modelo 60-30-10, o tempo caiu para 12 min, flakiness ficou abaixo de 2% e bugs em produção reduziram.



Ponto-chave

Mais testes unitários garantem deploys rápidos e seguros.

Verificação de Conhecimento: Pirâmide

Por que a base da Pirâmide de Testes deve ser composta majoritariamente por testes unitários?

1. Porque são os únicos capazes de testar a interface gráfica completa no navegador real.
2. Porque são extremamente rápidos de executar, baratos de manter e isolam a causa raiz das falhas.
3. Porque eliminam completamente a necessidade de testar a integração entre componentes.
4. Porque exigem ambientes de produção configurados com banco de dados em tempo real.

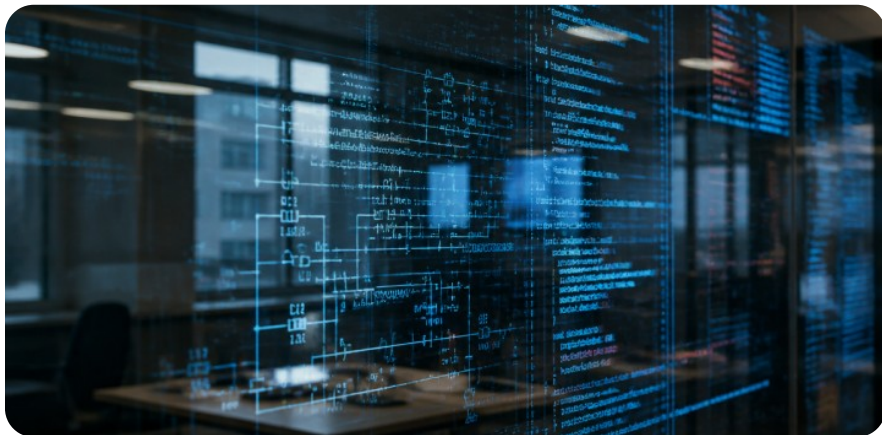
Verificação de Conhecimento: Pirâmide



Por que a base da Pirâmide de Testes deve ser composta majoritariamente por testes unitários?

1. Porque são os únicos capazes de testar a interface gráfica completa no navegador real.
2. Porque são extremamente rápidos de executar, baratos de manter e isolam a causa raiz das falhas.
3. Porque eliminam completamente a necessidade de testar a integração entre componentes.
4. Porque exigem ambientes de produção configurados com banco de dados em tempo real.

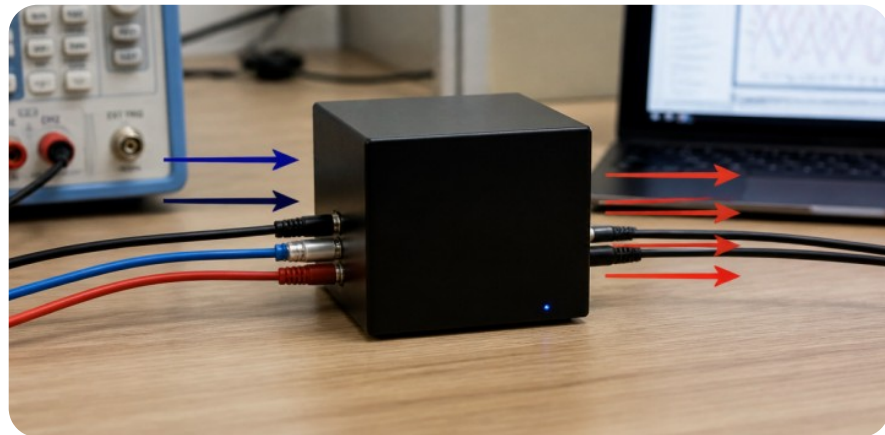
Abordagens: Caixa Branca vs Caixa Preta



Teste de Caixa Branca

Analisa código e arquitetura.

- Avalia lógica, branches e loops.
- Comum em testes unitários.
- Foca em cobertura e robustez.



Teste de Caixa Preta

Analisa entradas e saídas; ignora implementação.

- Foca em requisitos e usuário.
- Aplica-se a testes funcionais e E2E.
- Valida comportamento, não código.

A Regra de 10 de Myers: Custo de Bugs

ECONOMIA DO SOFTWARE

Custo cresce por etapa

Defeitos tardios custam exponencialmente mais:

Design: 1x (Base)

Dev: 10x

Testes: 100x

Produção: 1000x



Atenção

Produção: soma suporte, danos à reputação, multas e churn.



Casos Reais: O Custo de Ignorar Testes

Knight Capital (2012)

Um erro de deploy em um algoritmo financeiro operou sem validação por 45 minutos. O sistema comprou caro e vendeu barato, gerando prejuízo de **440 milhões de dólares** e falência quase imediata.



Atenção

Faltaram testes de regressão no deploy.

Shopify Black Friday

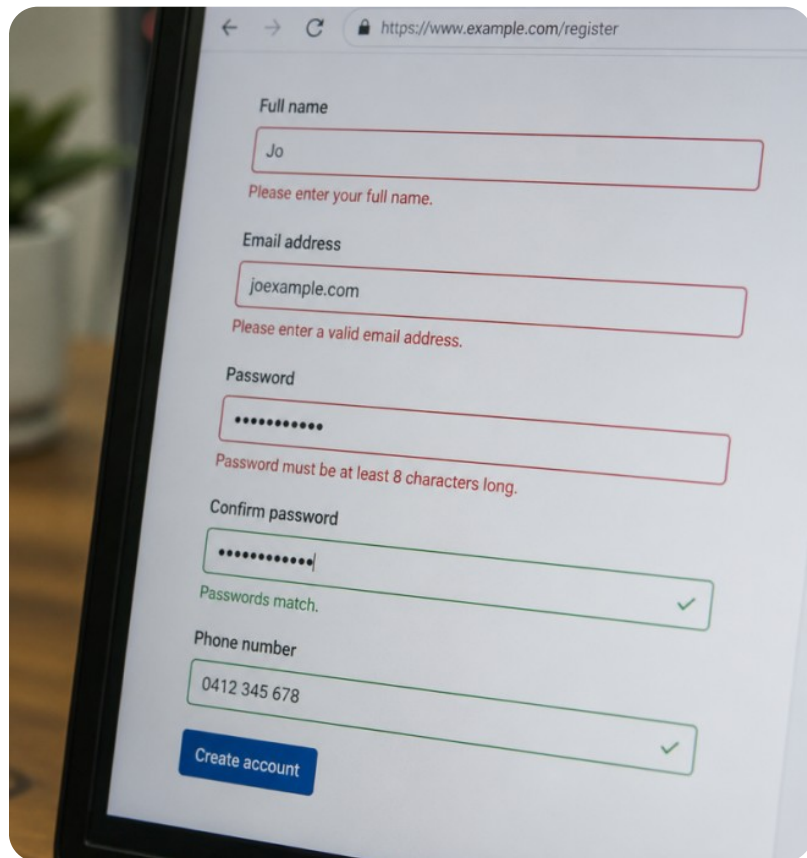
Uma condição de corrida em requisições de checkout derrubou serviços por 2 horas na Black Friday. Estimou-se perda de **500 mil dólares** por hora. A solução foi implementar testes de integração concorrentes.



Ponto-chave

Testes de carga garantem estabilidade sob picos de tráfego.

Testes Funcionais na Web Moderna



TIPOS DE REQUISITOS

Validando o comportamento do sistema

Testes funcionais verificam se o sistema segue as regras de negócio:

- Login e logout.
- Cálculo de frete.
- Filtros de produtos.



Exemplo

Exemplo: Email sem '@' dispara erro.

Testes Não-Funcionais na Web

TIPOS DE REQUISITOS

Validando o sistema

Qualidade exige acessibilidade, velocidade e segurança.

Performance: Carga rápida (LCP < 2.5s).

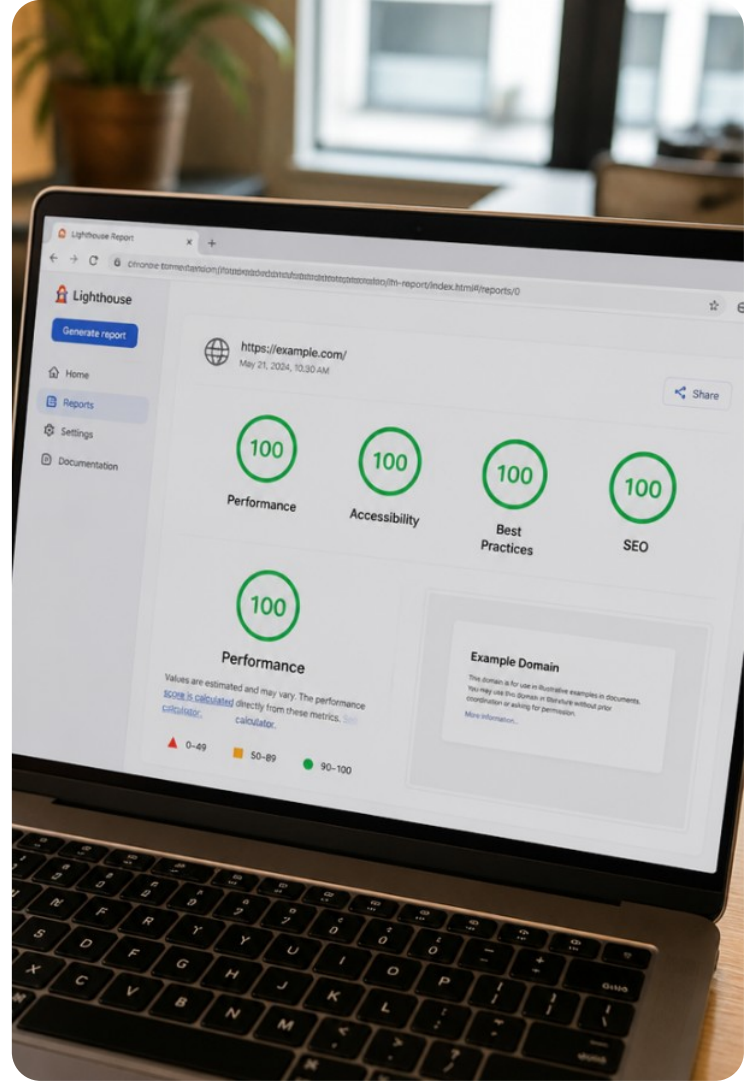
Acessibilidade: Teclado e leitores de tela.

Responsividade: Adaptação mobile e tablet.



Lembre-se

Página lenta (8s) afasta usuários.



Pilares de um Bom Teste Automatizado

1. Assertividade

O teste deve verificar o resultado esperado, sem falsos positivos. Deve falhar comunicando o motivo real da quebra.



Ponto-chave

Assertões devem ser claras: 'esperava 200, recebeu 404'.

2. Repetibilidade

Executar o teste 1 ou 1.000 vezes deve gerar o mesmo resultado. Testes imunes a variações de rede são confiáveis.



Atenção

Testes intermitentes (flaky) destroem a confiança da equipe.

3. Independência

Nenhum teste pode depender de outro. Cada teste prepara seu próprio estado e limpa os dados após concluir.



Lembre-se

Se o teste B depende do A, a falha do A mascara o B.

Mitos de Testes: Cobertura de 100%

Uma aplicação com 100% de cobertura de código em testes unitários está completamente livre de bugs em produção.



VERDADE
IRO



FALSO



Prepare-se para explicar o seu raciocínio.

Mitos de Testes: Cobertura de 100%



Uma aplicação com 100% de cobertura de código em testes unitários está completamente livre de bugs em produção.


VERDADE
IRO

 **FALSO** 



Por que é isso?

A cobertura unitária apenas prova que as linhas foram executadas, mas não valida requisitos esquecidos nem falhas na integração de múltiplos sistemas.

O Padrão AAA: Arrange, Act, Assert

BOAS PRÁTICAS

Estrutura canônica de testes

Todo teste deve seguir três etapas:

Arrange: Configura mocks e estado.

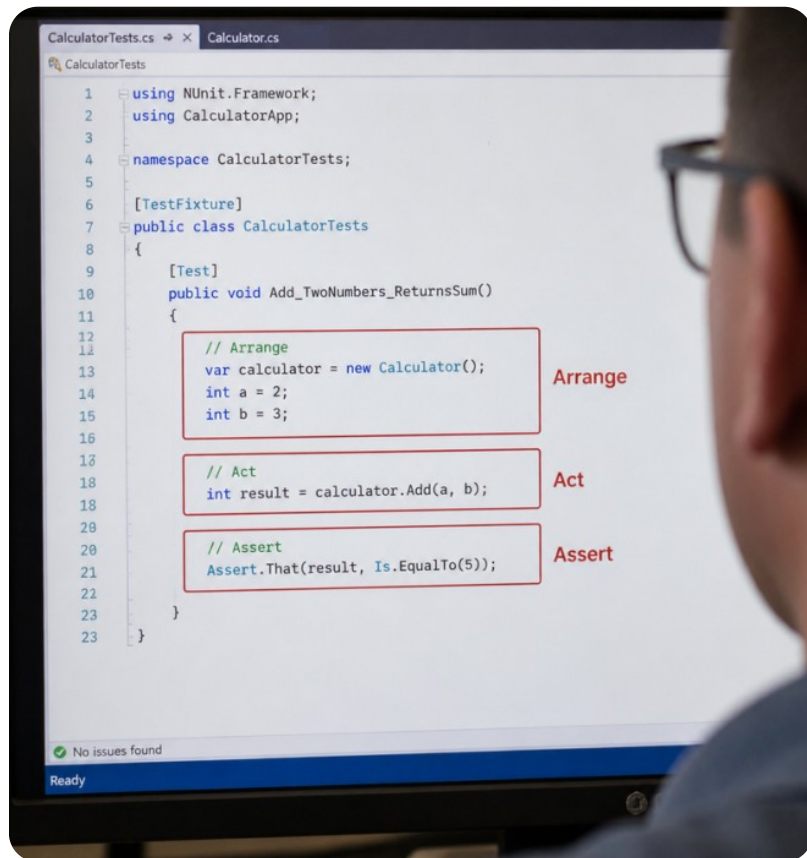
Act: Executa a função avaliada.

Assert: Valida o resultado obtido.



Lembre-se

Misturar etapas dificulta depuração.



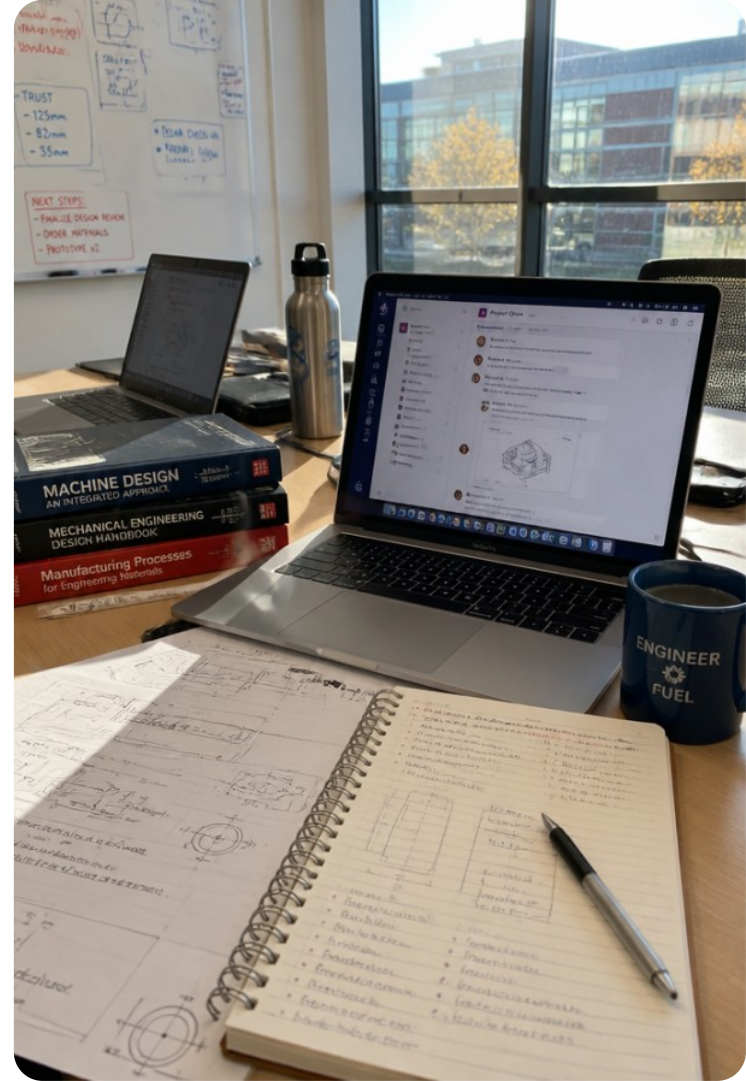
Entendendo Testes de Software na Prática

QA garante estabilidade e deploys seguros.



Atividades Práticas de Aplicação

Agora é sua vez de colocar a teoria à prova. Vamos exercitar a tomada de decisão técnica em cenários reais de engenharia de software.



Prática 1: Classificação de Cenários (1-5)

Classifique estes cenários na Pirâmide de Testes:

- 1** **Cenário 1:** Função calculaDesconto() (compras > R\$ 200) → Unitário.
- 2** **Cenário 2:** Validação máscara CPF durante digitação → Unitário.
- 3** **Cenário 3:** Envio formulário via API mockada (MSW) → Integração.
- 4** **Cenário 4:** Renderização MenuLateral com estado global → Integração.

Prática 1: Classificação de Cenários (6-10)



Classificação dos cenários 6-10:

1 **Cenário 6** - Conversão data ISO/BR: Unitário.

2 **Cenário 7** - Paginação/ordenação URL: Integração.

3 **Cenário 8** - Fluxo recuperação senha: E2E.

4 **Cenário 9** - Validação força senha: Unitário.

Fixação de Conceitos Chave

Relacione as palavras com as definições

Teste Unitário

Verificação da comunicação entre múltiplos módulos, componentes ou APIs.

Teste de Integração

Validação isolada e ultrarrápida de funções ou lógicas atômicas (<100ms).

Teste E2E

Simulação do fluxo completo do usuário através de navegadores automatizados.

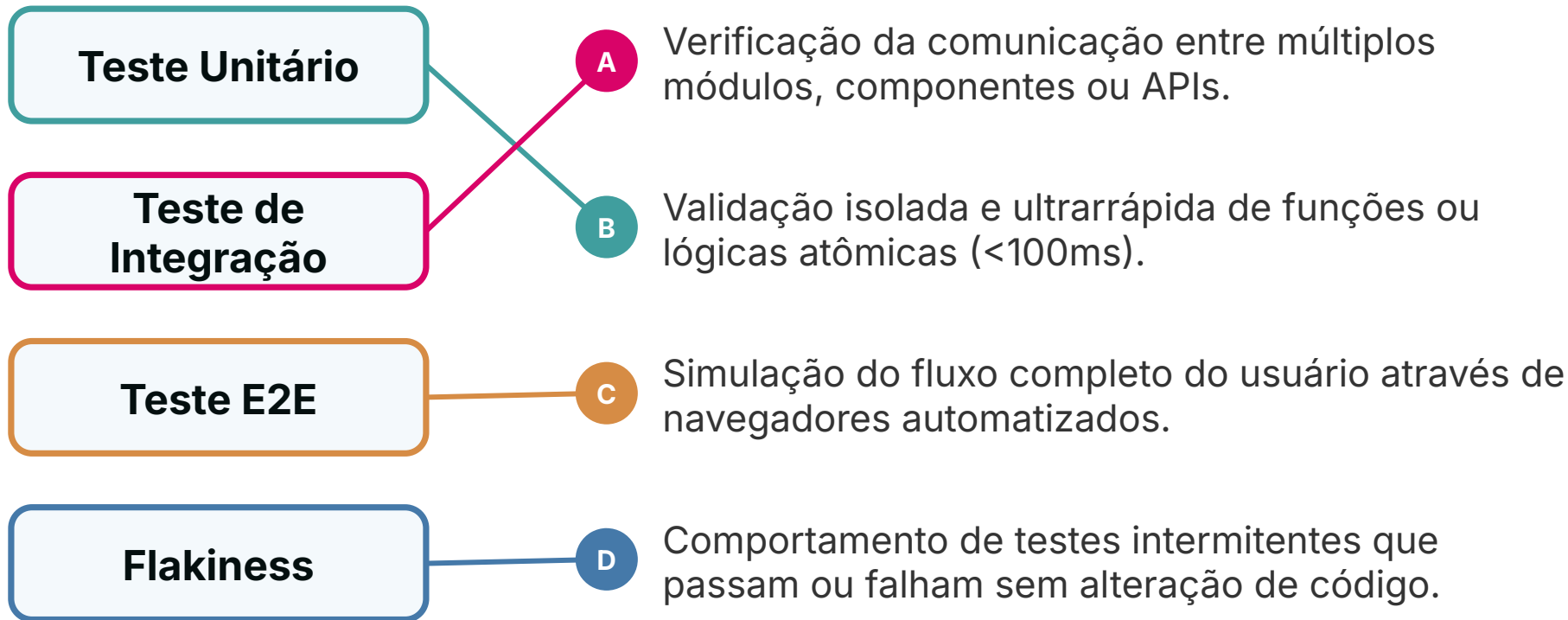
Flakiness

Comportamento de testes intermitentes que passam ou falham sem alteração de código.

Fixação de Conceitos Chave



Relacione as palavras com as definições



Prática 2: Pirâmide para Duas Aplicações

App 1: Fintech de Crédito

65% Unitários: Regras fiscais, juros e algoritmos de scoring.

25% Integração: Tokens JWT, serviços do Banco Central e extratos.

10% E2E: Jornadas de Pix e simulação de empréstimo.



Ponto-chave

Precisão matemática exige base unitária massiva.

App 2: E-commerce de Varejo

55% Unitários: Cálculos de frete, moeda e validadores.

30% Integração: Filtros de catálogo, busca e checkout.

15% E2E: Fluxos de conversão e login social.



Lembre-se

Interface rica demanda maior peso de integração.

Ordem de Execução no Pipeline

Ordene as etapas recomendadas para execução em um pipeline ágil de CI/CD, do teste mais rápido ao mais lento:

Liberação do deploy automatizado para o ambiente de produção

Execução da suite de testes unitários rápidos (<100ms por teste)

Execução dos testes de integração de componentes com mocks de API

Execução da suite de testes End-to-End (E2E) em ambiente de staging

Ordem de Execução no Pipeline

Ordene as etapas recomendadas para execução em um pipeline ágil de CI/CD, do teste mais rápido ao mais lento:



1
Execução da suite de testes unitários rápidos (<100ms por teste)

2
Execução dos testes de integração de componentes com mocks de API

3
Execução da suite de testes End-to-End (E2E) em ambiente de staging

4
Liberação do deploy automatizado para o ambiente de produção

Introdução ao Planejamento de Testes

PRÓXIMOS PASSOS

Planejamento (Bloco 2)

Na próxima aula, focaremos no STLC: **planejamento e formalização**.

- Verificação (Dev) vs. Validação (QA).
- Estrutura de Caso de Teste (ID, passos, pré-requisitos).
- Rastreabilidade com critérios BDD (Given/When/Then).



Lembre-se

Engenheiros definem o que validar antes de codar.



Síntese dos Fundamentos da Aula

Estratégia de Qualidade

- QA moderno é automatizado, preventivo e contínuo.
- O STLC orienta planejamento, design, execução e avaliação sistemática.
- Evite o 'cone de sorvete': mantenha a base unitária sólida.



Lembre-se

Qualidade é construída durante a escrita do código.

Equilíbrio Técnico

60% Unitários: Rápidos, pontuais e isolados.

30% Integração: Conexão entre componentes e APIs.

10% E2E: Confirmação da experiência real do usuário.



Ponto-chave

Respeitar AAA garante testes sustentáveis por anos.

Complete a Definição da Pirâmide

A Pirâmide de Testes recomenda que a maior parte dos testes sejam ____ devido à sua alta velocidade e baixo custo, seguidos por testes de ____ e, no topo, uma fração reduzida de testes ____.

Banco de palavras 

integração

redundantes

visuais

unitários

manuais

end-to-end

Complete a Definição da Pirâmide



A Pirâmide de Testes recomenda que a maior parte dos testes sejam **unitários** devido à sua alta velocidade e baixo custo, seguidos por testes de **integração** e, no topo, uma fração reduzida de testes **end-to-end**.

Banco de palavras 🏛️

integração

redundantes

visuais

unitários

manuais

end-to-end

Debate Crítico: Automação vs Manual



Em um time com prazo apertado para lançar uma nova funcionalidade de e-commerce, o gerente sugere pular a escrita de testes unitários e fazer apenas testes manuais rápidos no final. Como você argumentaria sobre os riscos técnicos e financeiros dessa decisão?

Debate Crítico: Automação vs Manual



Você poderia ter dito...

Pular testes unitários gera falsa rapidez inicial.

Refatorações futuras quebrarão comportamentos sem aviso.

Corrigir bugs em produção custa 100x mais que testar no dev.

Testes manuais não são repetíveis e sobrecarregam o time.

Desafio de Fixação Final



Pergunta 1:

Qual é a principal vantagem dos testes unitários sobre os testes end-to-end?

Pergunta 2:

O que significa o 'A' final no padrão AAA de testes?

Pergunta 3:

Qual fenômeno indesejado ocorre quando uma suite tem excesso de testes E2E e poucos testes unitários?

Desafio de Fixação Final



Resposta 1:

Velocidade de execução extrema ($<100\text{ms}$) e precisão cirúrgica no isolamento da causa raiz do erro.

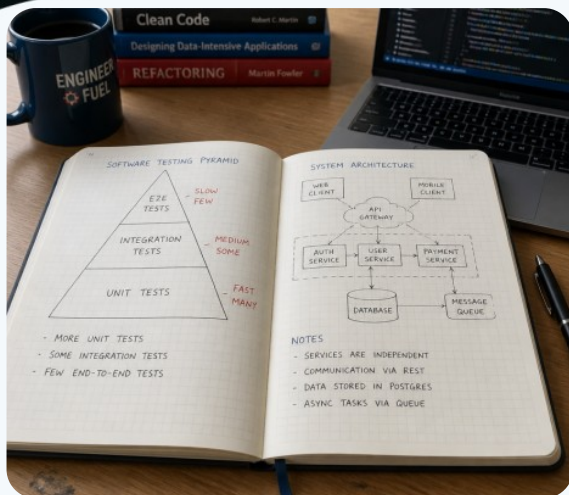
Resposta 2:

Assert (Afirmar/Verificar), que valida se o resultado produzido pelo código corresponde ao resultado esperado.

Resposta 3:

A pirâmide invertida (cone de sorvete), gerando lentidão extrema de pipeline e alta taxa de falhas intermitentes (flakiness).

Missão Prática: Desenhando sua Estratégia



Aplicativo: Spotify Web

Testes Unitários: Validar e-mail no login, calcular duração de playlist e verificar alternância do botão play.

Testes de Integração: Busca consulta API de tracks; botão skip dispara requisição ao backend.

Jornada End-to-End: Usuário busca música, adiciona à playlist e reproduz com sucesso.